

INSTITUTO FEDERAL

Catarinense
Campus Videira

Fundamentos do Processamento Digital de Imagens - PDI

Aula 02 - Formação da Imagem

Prof. Fabricio Bizotto
fabricio.bizotto@ifc.edu.br

Roteiro

- Definições básicas
- Processo de Geração das Imagens Digitais
- Formação da Imagem
- Representação da Imagem
- Processo de formação de cores
- O que é uma imagem?
- Criando e manipulando imagens (exemplos práticos)
- Experimentos (exercícios propostos)

Definições básicas

- Uma imagem digital é uma representação de uma cena por meio de um conjunto de elementos discretos e de tamanhos finitos, chamados de pixels, colocados em um arranjo bidimensional. A cada pixel é associado um valor, no caso de imagens de tons de cinza, ou um conjunto de três valores RGB (Red-Green-Blue) para se representar uma cor.
- A imagem pode ser criada a partir de softwares de desenho ou adquirida por meio de algum dispositivo. A maneira como esta imagem é armazenada no computador define o formato do arquivo. Os formatos mais comuns são: bmp (bitmap), gif, jpg, png, tiff.
- A área de PDI se refere à manipulação destes pixels visando a melhoria na apresentação da imagem, o realce ou eliminação de certas características e a extração de informações.

Processo de Geração das Imagens Digitais

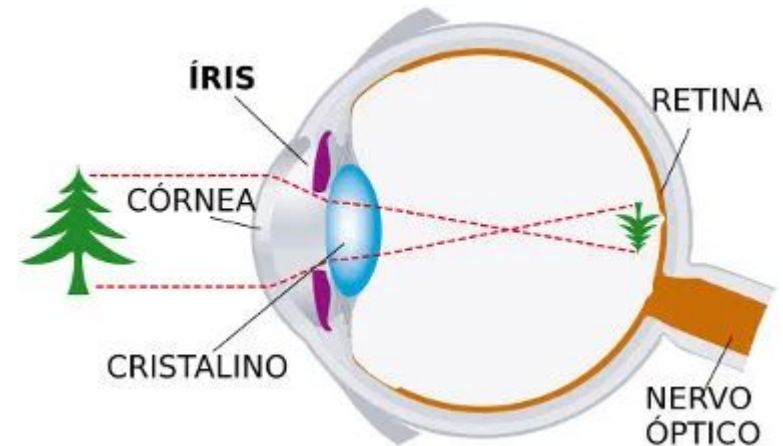
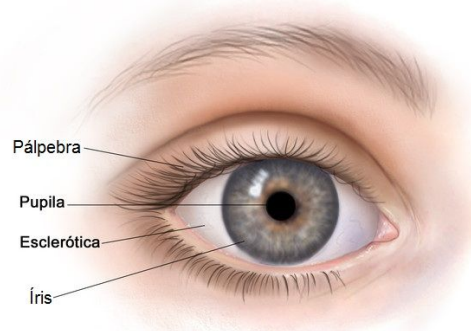
- As imagens digitais são geradas por um processo que podemos chamar de digitalização. O processo de digitalização possibilita que sinais analógicos, provenientes da energia que chega ao sensor do digitalizador, sejam transformados em informação digital.



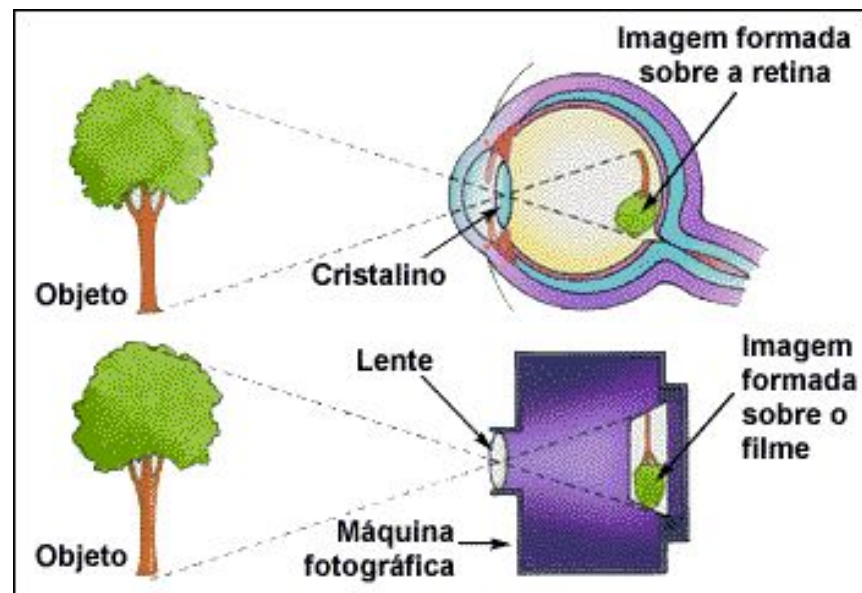
Formação da Imagem

Os raios luminosos provenientes dos objetos penetram no olho através de uma abertura frontal na **íris**, a **pupila**, e de uma lente denominada **cristalino**, atingindo a **retina**, que constitui a camada interna posterior do globo ocular.

- **Íris**: controla abertura do olho
- **Pupila**: abre e fecha para entrar luz
- **Cristalino**: Funciona como a lente do olho
- **Retina**: Plano de formação da imagem
- **Nervo óptico**: Transmissor

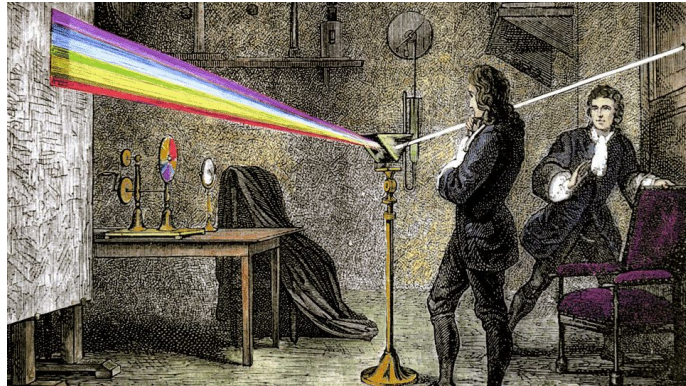


Formação da Imagem



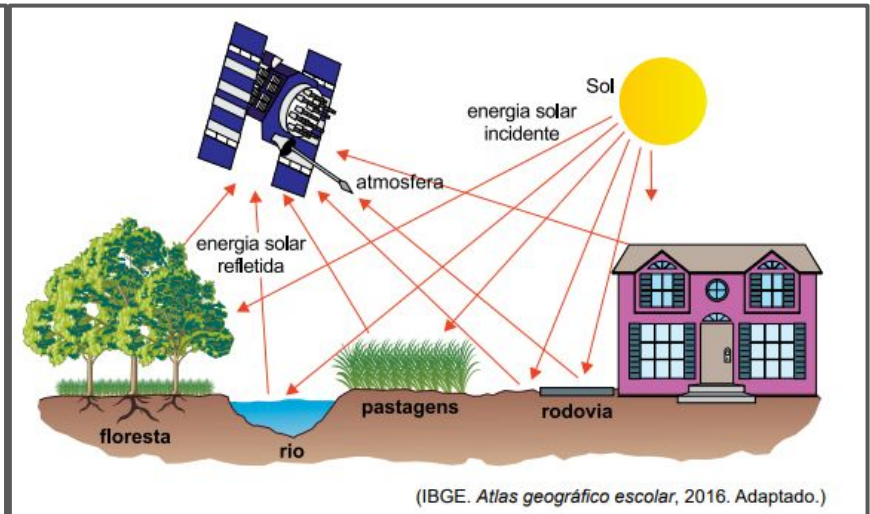
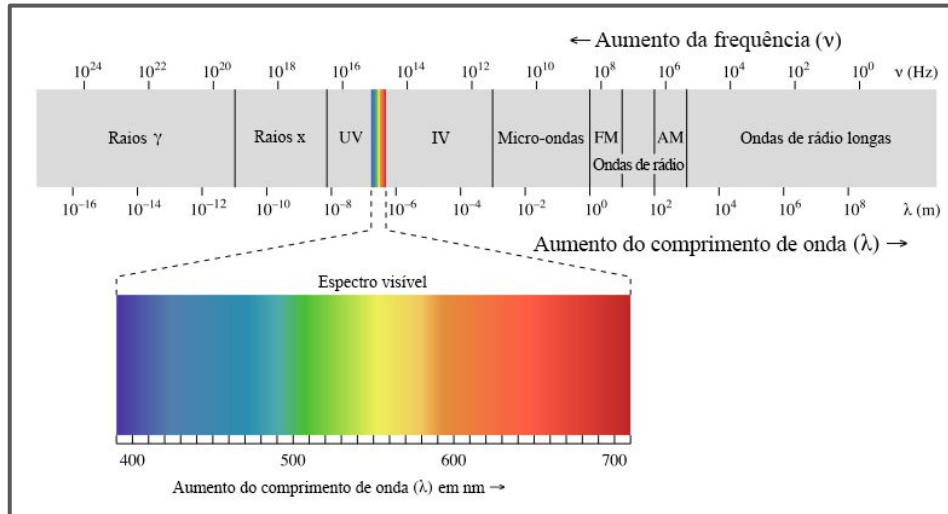
Processo de formação de cores

- Isaac Newton realizou experimentos com um prisma de vidro e observou que a luz branca, ao passar pelo prisma, se dividia em um espectro de cores, que ia do vermelho ao violeta.
- Ele propôs que as cores são a resultante de diferentes frequências da luz visível, que estimulam diferentes células sensíveis à luz no olho humano.
- Segundo Newton, cada cor é associada a uma determinada frequência de onda, sendo o vermelho a cor com a menor frequência e o violeta a cor com a maior frequência. Ele também percebeu que a mistura das cores primárias (vermelho, azul e amarelo) pode produzir outras cores.



Processo de formação de cores

- Absorção da luz é um fenômeno óptico que ocorre quando a radiação eletromagnética visível incide sobre a superfície de algum material, de forma que alguma parcela da energia carregada por essa luz permaneça retida nele. As ondas eletromagnéticas podem ser classificadas e organizadas de acordo com seus diversos comprimentos de onda/frequências.
- O olho humano é sensível à radiação eletromagnética na faixa de 400 a 700 nanômetros, chamada espectro visível, dentro da qual estão localizadas as chamadas sete cores visíveis.

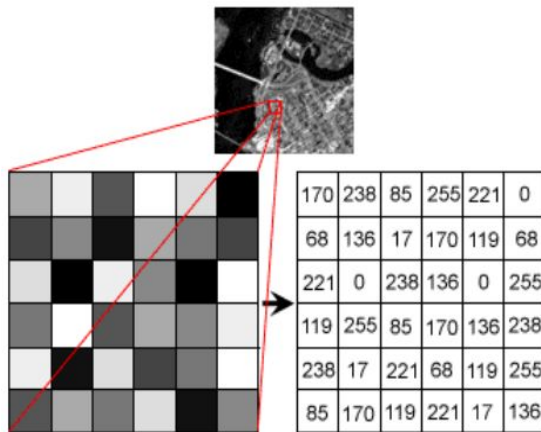
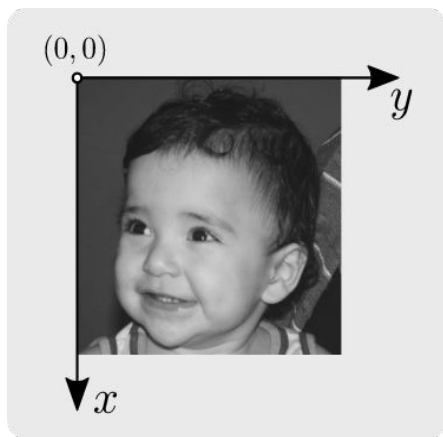


Material Complementar

- Ondas eletromagnéticas e espectro eletromagnético
- Polarização da luz, linear e circular

O que é uma imagem?

- Uma imagem é uma função bidimensional de intensidade da luz $f(x,y)$, onde x e y denotam as coordenadas espaciais e o valor de f em qualquer ponto (x,y) é proporcional ao brilho da imagem nesse ponto.
- Uma imagem digital pode ser considerada como uma matriz cujos índices de linhas e colunas identificam um ponto na imagem.
- Os elementos dessa matriz são chamados de **pixels** (abreviação de picture elements)



Sistema de coordenadas da imagem

		Colunas (Y) →				
Linhas (X) ↓	56	55	55	57	54	Pixel (2, 2)
	55	55	57	55	55	
	78	81	81	82	80	
	80	82	79	78	77	
	82	82	84	84	85	

Representação

- Uma imagem é uma função bidimensional de intensidade da luz $f(x,y)$, onde x e y denotam as coordenadas espaciais.
- O valor de f em qualquer ponto (x,y) é proporcional ao brilho da imagem nesse ponto.
- A natureza dos tons da imagem pode ser caracterizada por dois componentes: intensidade luminosa refletida pelo material $i(x,y)$, que depende da fonte de energia e da reflectância $r(x,y)$, que depende das propriedades do material

$$f(x,y) = i(x,y) r(x,y)$$

onde $0 < i(x,y) < \infty$ e $0 < r(x,y) < 1$

- $r(x,y) \approx 1$, tende a refletir a luz (ex: cor branca)
- $r(x,y) \approx 0$, tende a absorver a luz (ex: cor preta)



Imagem monocromática e policromática

- Uma **imagem monocromática** é composta por apenas uma cor, normalmente preto e branco, ou uma variação de tons cinza. Essas imagens são produzidas em escala de cinza, onde a intensidade do preto varia de acordo com a luminosidade de cada pixel da imagem.
 - Para imagens monocromáticas, apenas uma matriz é usada
- Já as **imagens policromáticas**, também conhecidas como imagens em cores, são compostas por duas ou mais cores. Essas imagens podem ser produzidas em diversos formatos de cores, como RGB (vermelho, verde e azul) ou CMYK (ciano, magenta, amarelo e preto), e podem apresentar uma grande variedade de tonalidades e matizes.
 - Normalmente, várias matrizes são necessárias para representar a noção de cor.
 - Imagens de satélite geralmente armazenam vários canais ou bandas

Imagem monocromática e policromática

- As imagens podem conter informação de cor ou não (imagem binária).
- Para imagens monocromáticas, apenas uma matriz é usada e a imagem é representada em escala de cinza (*grayscale*).
- Para imagens policromáticas (coloridas **Red**, **Green**, **Blue**), três componentes de cor (ou matrizes) são utilizadas.



Amostragem e Quantização

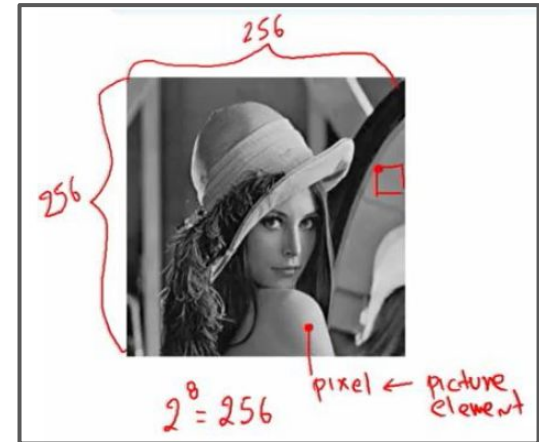
- Chamaremos de **amostragem** o processo de discretização espacial e daremos o nome de quantização ao processo de discretização em amplitude.
- Basicamente, a amostragem converte a imagem analógica em uma matriz de M por N pontos, cada qual denominado pixel.

$$f(x, y) = \begin{bmatrix} f(0,0) & f(0,1) & \dots & f(0, N-1) \\ f(1,0) & f(1,1) & \dots & f(1, N-1) \\ \vdots & \vdots & \vdots & \vdots \\ f(M-1,0) & f(M-1,1) & \dots & f(M-1, N-1) \end{bmatrix}$$

Amostragem e Quantização

$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & \dots & f(0,N-1) \\ f(1,0) & f(1,1) & \dots & f(1,N-1) \\ \vdots & \vdots & \vdots & \vdots \\ f(M-1,0) & f(M-1,1) & \dots & f(M-1,N-1) \end{bmatrix}$$

- Chamaremos de **amostragem** o processo de discretização espacial e daremos o nome de quantização ao processo de discretização em amplitude.
- Basicamente, a amostragem converte a imagem analógica em uma matriz de M por N pontos, cada qual denominado pixel.
- A **quantização** faz com que cada um destes pixels assuma um valor inteiro, na faixa de **[0 a $2^n - 1$]**. Quanto maior o valor de n, maior o número de níveis de cinza presentes na imagem.
 - Caso $n=1$: imagem binária (“preto e branco”);
 - Caso $n>1$: imagem “tons de cinza”;
 - Caso $n=8$: cada pixel pode representar até 256 tons de cinza, suficiente para distinção pelo olho humano;



Efeito da redução de elementos na matriz de pontos

- Redução espacial da imagem pode causar perda de detalhes;
- Efeito “tabuleiro de xadrez”;



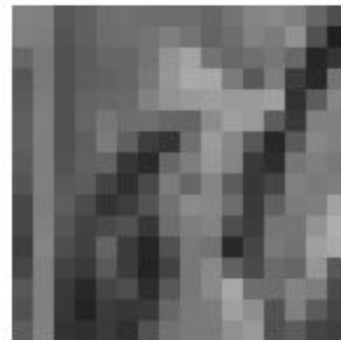
128x128



64x64



32x32



16x16

Efeito da redução da quantidade de bits da imagem

- Visível em imagens com 16 tons de cor ou menos ($2^{4\text{bits}} = 16 \text{ tons de cor}$)
- Efeito “falso contorno”

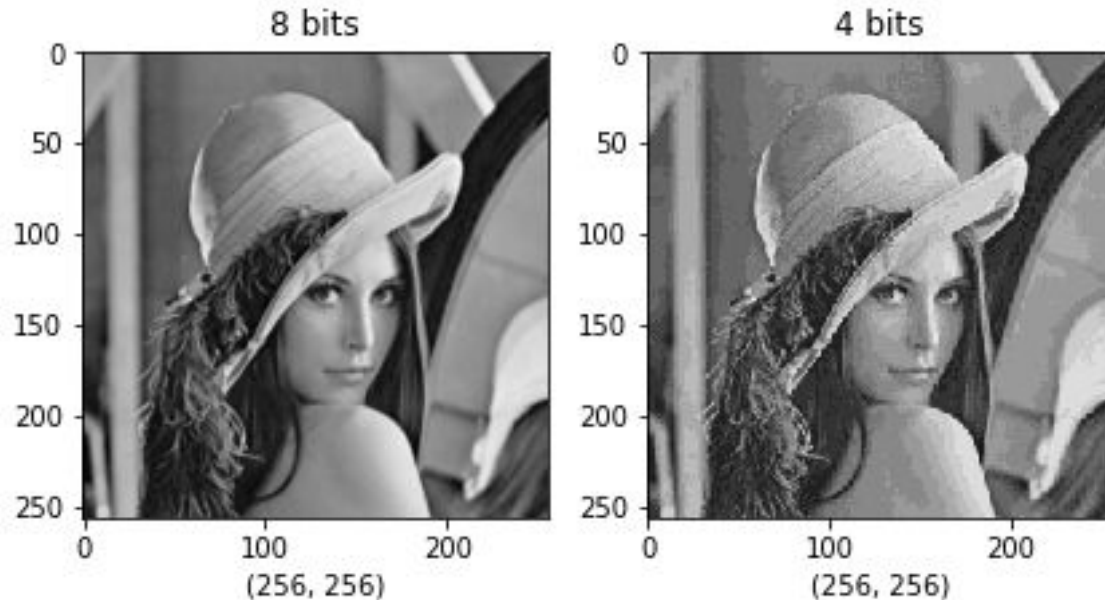


Imagem binária

- As imagens podem conter informação de cor ou não (imagem binária).



0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	1	1	1
0	0	0	0	1	1	1
0	0	0	0	1	1	1
0	0	0	0	1	1	1

Algumas aplicações em que as imagens binárias são úteis:

- **Reconhecimento de caracteres:** Em sistemas de reconhecimento óptico de caracteres (OCR), as imagens binárias são usadas para separar o texto do fundo da página.
- **Detecção de bordas:** As imagens binárias podem ser usadas para detectar bordas em imagens. Isso é útil em aplicações como detecção de bordas em radiografias ou detecção de contornos em imagens médicas.
- **Segmentação de objetos:** A segmentação de objetos é uma técnica em que uma imagem é dividida em várias regiões, cada uma delas contendo um objeto. As imagens binárias são frequentemente usadas na segmentação de objetos, pois permitem que a região de interesse seja claramente definida.

Escala de Cinza (grayscale)

Aplicações em que as imagens em escala de cinza são úteis

- Processamento de imagens médicas: As imagens em escala de cinza são amplamente usadas em imagens médicas, como tomografias e ressonâncias magnéticas, para visualizar tecidos e órgãos internos do corpo.

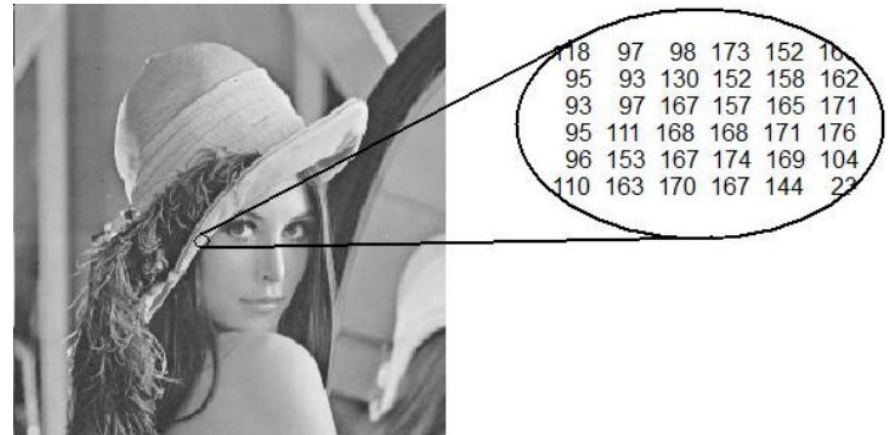


Imagem Colorida - True Color

- Utiliza 24 bits por pixel com 8 bits para cada cor que possibilita 256 níveis de intensidade e um total de 16 milhões de cores por pixel.
- O valor (0,0,0) de RGB equivale ao preto, e o valor (255,255,255) de RGB equivale ao branco



RED						
175	177	175	184	178	176	
177	174	179	175	178	179	
174	175	178	177	175	183	
			174	179	183	
			177	177	175	
			176	176	177	

GREEN						
72	63	63	62	64	72	
69	63	63	67	62	66	
67	56	65	64	62	67	
				63	69	
				63	65	
				66	63	

BLUE						
81	74	78	72	76	79	
73	70	72	75	76	72	
71	71	71	78	76	77	
79	73	72	78	70	76	
77	73	71	72	76	74	
81	72	73	80	83	75	

Imagem Colorida - False Color

- Em contraste com uma imagem de cor verdadeira, uma imagem de cor falsa sacrifica a reprodução natural de cores para facilitar a detecção de características que não são facilmente discerníveis de outra forma - por exemplo, o uso de infravermelho próximo para a detecção de vegetação em imagens de satélite



True Color

False Color

Quiz

Questão 01

Qual é a profundidade de uma imagem com 65536 níveis de cinza?

Questão 01

Qual é a profundidade de uma imagem com 65536 níveis de cinza?

16 bits

Isso significa que cada pixel na imagem pode ter um valor de intensidade de cinza entre 0 e 65535 (ou $2^{16} - 1$). Uma imagem com essa profundidade é capaz de capturar uma ampla gama de tonalidades de cinza e é comumente usada em aplicações de processamento de imagem que requerem alta precisão na representação de detalhes sutis em imagens em escala de cinza.

Questão 02

Qual é a profundidade de uma imagem RGB?

Questão 02

Qual é a profundidade de uma imagem RGB?

Geralmente 24 bits

Significa que cada pixel é composto por três componentes de cor (vermelho, verde e azul) e cada um desses componentes tem uma profundidade de 8 bits. Cada componente de cor pode ter um valor de intensidade entre 0 e 255 (ou $2^8 - 1$), permitindo assim a representação de uma ampla gama de cores na imagem. Além disso, existem outras profundidades de cores possíveis para imagens RGB, como 16 bits por canal, o que aumentaria a precisão na representação de cores sutis e nuances.

Questão 03

Qual é a diferença entre imagem binária, imagem monocromática e imagem colorida?

Questão 03

Qual é a diferença entre imagem binária, imagem monocromática e imagem colorida?

Em resumo, enquanto a imagem binária tem apenas duas opções de cor (**preto ou branco**), a imagem monocromática tem vários **níveis de cinza** e a imagem colorida tem informações de cor em adição às informações de níveis de intensidade de luz. Cada pixel em uma imagem colorida é representado por três valores de intensidade de cor: RGB.

Questão 04

Quais são as diferenças entre RGB e HSL?

Questão 04

Quais são as diferenças entre RGB e HSL?

RGB e HSL são dois modelos de cores diferentes usados na representação de imagens digitais.

Em resumo, enquanto o modelo RGB representa as cores como misturas de luz em diferentes intensidades de vermelho, verde e azul, o modelo HSL representa as cores como matizes, saturação e luminosidade. Cada modelo é mais adequado para diferentes tipos de aplicações, dependendo das necessidades específicas de representação de cores.

Criando e Manipulando Imagens

<https://colab.research.google.com>

```
# Exemplo: Criando uma imagem grayscale
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Tamanho da imagem (largura x altura)
```

```
w = h = 8
```

```
# Criando uma imagem cinza de 8x8 pixels
```

```
gray = np.arange(0, w*h, 1, np.uint8)
```

```
print(gray)
```

```
gray = np.reshape(gray, (w,h))
```

```
print(gray)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63]
```

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]
 [16 17 18 19 20 21 22 23]
 [24 25 26 27 28 29 30 31]
 [32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47]
 [48 49 50 51 52 53 54 55]
 [56 57 58 59 60 61 62 63]]
```



```
# Exemplo: Criando uma imagem grayscale
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Tamanho da imagem (largura x altura)
w = h = 8
```

```
# Criando uma imagem cinza de 8x8 pixels
gray = np.arange(0, w*h, 1, np.uint8)
print(gray)
```

```
gray = np.reshape(gray, (w,h))
print(gray)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63]
```

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]
 [16 17 18 19 20 21 22 23]
 [24 25 26 27 28 29 30 31]
 [32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47]
 [48 49 50 51 52 53 54 55]
 [56 57 58 59 60 61 62 63]]
```

```
fig, ax = plt.subplots(1, 1, figsize=(9,3))
ax.set_title(f'Grayscale | {gray.shape}')
ax.imshow(gray, cmap='gray')
```

```
plt.tight_layout() # clear
plt.show()
```

```
# Exemplo: Criando uma imagem grayscale
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Tamanho da imagem (largura x altura)
w = h = 8
```

```
# Criando uma imagem cinza de 8x8 pixels
```

```
gray = np.arange(0, w*h, 1, np.uint8)
print(gray)
```

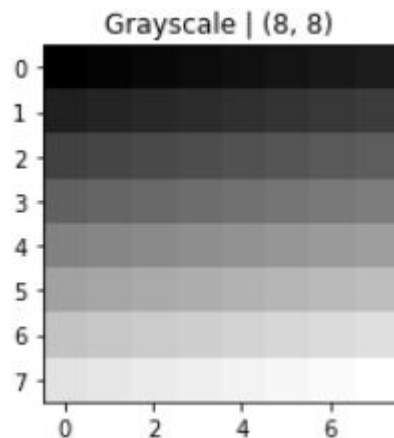
```
gray = np.reshape(gray, (w,h))
print(gray)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63]
```

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]
 [16 17 18 19 20 21 22 23]
 [24 25 26 27 28 29 30 31]
 [32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47]
 [48 49 50 51 52 53 54 55]
 [56 57 58 59 60 61 62 63]]
```

```
fig, ax = plt.subplots(1, 1, figsize=(9,3))
ax.set_title(f'Grayscale | {gray.shape}')
ax.imshow(gray, cmap='gray')
```

```
plt.tight_layout() # clear
plt.show()
```



```
# Exemplo: Criando uma imagem grayscale
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Tamanho da imagem (largura x altura)
w = h = 8
```

```
# Criando uma imagem cinza de 8x8 pixels
```

```
gray = np.arange(0, w*h, 1, np.uint8)
print(gray)
```

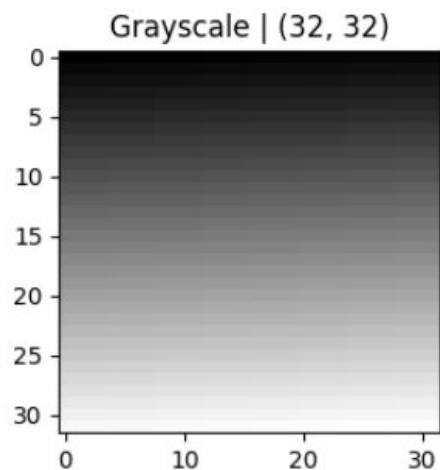
```
gray = np.reshape(gray, (w,h))
print(gray)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63]
```

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]
 [16 17 18 19 20 21 22 23]
 [24 25 26 27 28 29 30 31]
 [32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47]
 [48 49 50 51 52 53 54 55]
 [56 57 58 59 60 61 62 63]]
```

```
fig, ax = plt.subplots(1, 1, figsize=(9,3))
ax.set_title(f'Grayscale | {gray.shape}')
ax.imshow(gray, cmap='gray')
```

```
plt.tight_layout() # clear
plt.show()
```



```
# Alterações para aceitar mais que 8 bits
w = h = 32
gray = np.arange(0, w*h, 1, np.uint64)
```



```
# Exemplo: Criando uma imagem grayscale
import numpy as np
import matplotlib.pyplot as plt

# Tamanho da imagem (largura x altura)
w = h = 8

# Criando imagem RGB com valores aleatórios entre 0 e 255
rgb = np.array([
    np.random.randint(
        255,
        size=(w,h),
        dtype=np.uint8)
    for x in range(3)
])
print(rgb)
rgb = rgb.transpose(2,1,0)
print(rgb)
```



```
# Exemplo: Criando uma imagem grayscale
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Tamanho da imagem (largura x altura)
```

```
w = h = 8
```

```
# Criando imagem RGB com valores aleatórios entre 0 e 255
```

```
rgb = np.array([  
    np.random.randint(  
        255,
```

```
        size=(w,h),
```

```
        dtype=np.uint8)
```

```
    for x in range(3)
```

```
])
```

```
print(rgb)
```

```
rgb = rgb.transpose(2,1,0)
```

```
print(rgb)
```

```
[[[ 37 239 179  71 160  90  88 218]  
  [192 247 209 222 230 212  68 142]  
  [108  10 244 153 235 115 228 158]  
  [249 157 228 169 232 225  38 195]  
  [ 21 129 193  74  21 168 172  70]  
  [178 131  62  93 105  97 166 142]  
  [ 44 113 135 110 204 197 197 153]  
  [217 154  74 134 194  26  64  85]]]
```

```
[[208 163 229  9 210  80  88  73]  
 [122 106 107 105 252  71  13 139]  
 [ 35 152  83 227 167 100 245  13]  
 [237 207 190 117 216 165  96 166]  
 [ 73 203 154 240  48 220 178  49]  
 [ 10 203 201  41 121  37 239  87]  
 [144  21 190 220 220 192  20 121]  
 [ 34 254  33 198 192 225 196 167]]]
```

```
[[189 144 130 142  59  36  6 178]  
 [ 24 127  66 150 219  28 217  40]  
 [ 69 161 131 222 151  30  41 197]  
 [ 43 133 130 172 108  51  18  4]  
 [ 71 246 181  87 154 173 185 122]  
 [194 105  78  20  70 217 206 145]  
 [221 205 231  52  44  70 228 153]  
 [161 240 198  74  27 108  85 194]]]
```




```
# Exemplo: Criando uma imagem grayscale
import numpy as np
import matplotlib.pyplot as plt

# Tamanho da imagem (largura x altura)
w = h = 8

# Criando imagem RGB com valores aleatórios entre 0 e 255
rgb = np.array([
    np.random.randint(
        255,
        size=(w,h),
        dtype=np.uint8)
    for x in range(3)
])
print(rgb)
rgb = rgb.transpose(2,1,0)
print(rgb)
```

```
[[[ 37 239 179  71 160  90  88 218]
  [192 247 209 222 230 212  68 142]
  [108  10 244 153 235 115 228 158]
  [249 157 228 169 232 225  38 195]
  [ 21 129 193  74  21 168 172  70]
  [178 131  62  93 105  97 166 142]
  [ 44 113 135 110 204 197 197 153]
  [217 154  74 134 194  26  64  85]]

 [[208 163 229  9 210  80  88  73]
  [122 106 107 105 252  71  13 139]
  [ 35 152  83 227 167 100 245  13]
  [237 207 190 117 216 165  96 166]
  [ 73 203 154 240  48 220 178  49]
  [ 10 203 201  41 121  37 239  87]
  [144  21 190 220 220 192  20 121]
  [ 34 254  33 198 192 225 196 167]]

 [[189 144 130 142  59  36  6 178]
  [ 24 127  66 150 219  28 217  40]
  [ 69 161 131 222 151  30  41 197]
  [ 43 133 130 172 108  51  18  4]
  [ 71 246 181  87 154 173 185 122]
  [194 105  78  20  70 217 206 145]
  [221 205 231  52  44  70 228 153]
  [161 240 198  74  27 108  85 194]]]
```

```
[[[ 37 208 189]
  [192 122  24]
  [108  35  69]
  [249 237  43]
  [ 21  73  71]
  [178  10 194]
  [ 44 144 221]
  [217  34 161]]

 [[239 163 144]
  [247 106 127]
  [ 10 152 161]
  [157 207 133]
  [129 203 246]
  [131 203 105]
  [113  21 205]
  [154 254 240]]

 [[179 229 130]
  [209 107  66]
  [244  83 131]
  [228 190 130]]]
```

```

▶ # Exemplo: Criando uma imagem grayscale
import numpy as np
import matplotlib.pyplot as plt

# Tamanho da imagem (largura x altura)
w = h = 8

# Criando imagem RGB com valores aleatórios entre 0 e 255
rgb = np.array([
    np.random.randint(
        255,
        size=(w,h),
        dtype=np.uint8)
    for x in range(3)
])
print(rgb)
rgb = rgb.transpose(2,1,0)
print(rgb)

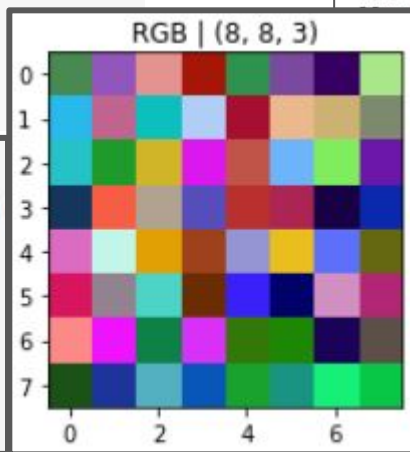
```

```

# Apenas com a média entre os canais
fig, ax = plt.subplots(1, 1, figsize=(9,3))
ax.set_title(f'RGB | {rgb.shape}')
ax.imshow(rgb)

plt.tight_layout() # clear
plt.show()

```



```

[[[ 37 239 179  71 160  90  88 218]
 [192 247 209 222 230 212  68 142]
 [108  10 244 153 235 115 228 158]
 [249 157 228 169 232 225  38 195]
 [ 21 129 193  74  21 168 172  70]
 [178 131  62  93 105  97 166 142]
 [ 44 113 135 110 204 197 197 153]
 [217 154  74 134 194  26  64  85]]]

```

```

[[208 163 229  9 210  80  88  73]
 [122 106 107 105 252  71  13 139]
 [ 35 152  83 227 167 100 245  13]
 [237 207 190 117 216 165  96 166]
 [ 73 203 154 240  48 220 178  49]
 [ 10 203 201  41 121  37 239  87]
 [144  21 190 220 220 192  20 121]
 [ 34 254  33 198 192 225 196 167]]]

```

```

144 130 142  59  36  6 178]
127  66 150 219  28 217  40]
161 131 222 151  30  41 197]
133 130 172 108  51  18  4]
246 181  87 154 173 185 122]
105  78  20  70 217 206 145]
205 231  52  44  70 228 153]
240 198  74  27 108  85 194]]]

```

```

[[[ 37 208 189]
 [192 122  24]
 [108  35  69]
 [249 237  43]
 [ 21  73  71]
 [178  10 194]
 [ 44 144 221]
 [217  34 161]]]

```

```

[[239 163 144]
 [247 106 127]
 [ 10 152 161]
 [157 207 133]
 [129 203 246]
 [131 203 105]
 [113  21 205]
 [154 254 240]]]

```

```

[[179 229 130]
 [209 107  66]
 [244  83 131]
 [228 190 130]]]

```



```
# Exemplo: Convertendo uma imagem RGB em Grayscale
import numpy as np
import matplotlib.pyplot as plt

# Tamanho da imagem (largura x altura)
w = h = 8

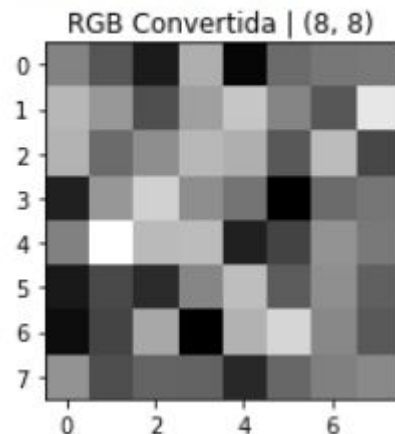
# Calculando manualmente
print(rgb[0][0]) # primeiros três valores RGB
print(sum(rgb[0][0])/3) # média dos três canais

# Calculando a média
# axis={0 para 1D, 1 para 2D, 2 para 3D}
rgb_to_gray = np.mean(rgb, axis=2)

# Apenas com a média entre os canais
fig, ax = plt.subplots(1, 1, figsize=(9,3))
ax.set_title(f'RGB Convertida | {rgb_to_gray.shape}')
ax.imshow(rgb_to_gray, cmap='gray')

plt.tight_layout() # clear
plt.show()
```

[37 208 189]
144.66666666666666



Experimentos

1. Ler um arquivo de imagem em escala de cinza, exibir, somar 100 unidades a cada pixel, exibir o resultado, comparar e gravar o novo arquivo.
2. Repetir o mesmo processo com uma imagem colorida
3. Repetir o mesmo processo para um conjunto de imagens gravadas em um diretório
4. Leia [este arquivo csv](#) e converta em uma imagem em escala de cinza. Grave a imagem em formato PNG